nod Stellar testnet

HackMeridian 2026 · Genesis · Stellar Composed

Give your AI an allowance.

Small stuff, it just pays. Big stuff needs your nod.

Live on Stellar testnet `testnet.trynod.si`

IN YOUR WALLET

30.00

USDC · Personal · testnet

CLAUDE MAY SPEND

5.00

USDC, until tomorrow 12:00 UTC

20.00 USDC to Bob

Claude asked · "Dinner with Bob"

Needs your nod



THE PROBLEM

Your AI can do everything except pay safely.

GIVE IT EVERYTHING

Hand over a card or a private key.

No limit, no expiry, no way to see what it may spend. One bad prompt can empty the account.

GIVE IT NOTHING

Make every payment yourself.

The task stops at checkout. A paid API call or paying a friend back waits for you.

> **“There is no policy layer for x402 on Stellar today.”**
>
> Stellar developer docs, on agent payments

THE PRODUCT

One connector. Two lanes.

CLAUDE ASKS TO PAY

Add Nod to Claude, or any MCP app, with one URL. Each AI app gets its own allowance on your Stellar account.

Inside the allowance, it just pays.

Paid APIs over x402 and contacts by name. Claude's own key signs; the account checks the limit.

```
0.10 · market brief
2.00 · Alice
```

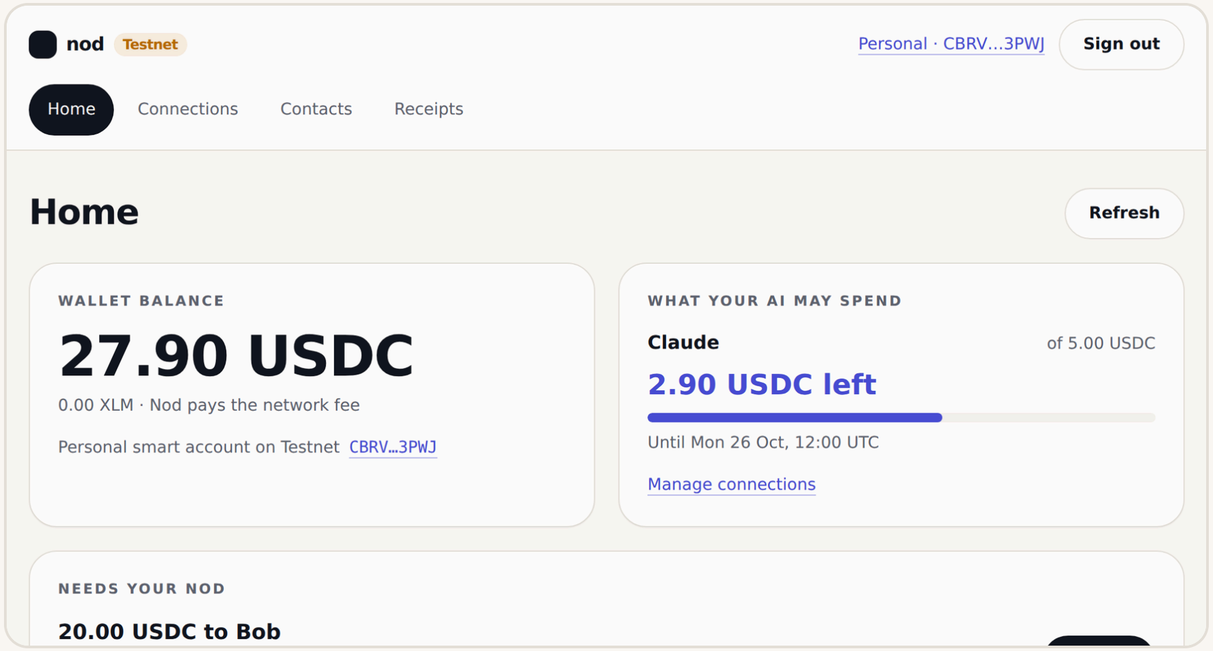
Over it, it needs your nod.

Claude gets a link. You see who, how much and why, and your passkey signs that exact payment.

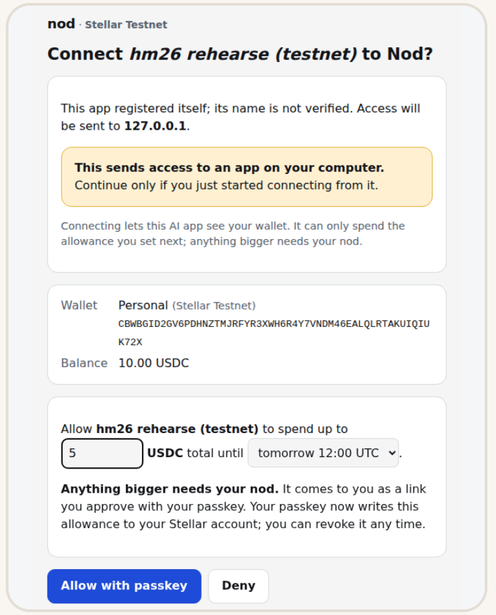
```
20.00 · Bob, dinner
Face ID → paid
```

DEMO

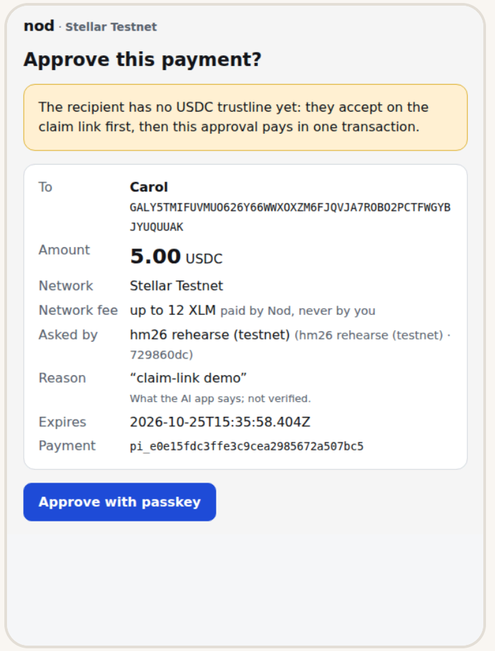
It works on Stellar testnet today.



Home: what you hold, what Claude may spend, what needs your nod (demo data)



Consent: the allowance, signed by your passkey



Approval: recipient, amount, who asked

0.10	**2.00**	**20.00**	**×2 → ×1**	**5.00**	**revoke**
Buys the market brief over x402	Pays Alice by name	Bob's dinner needs your nod	Ask twice, pay once	Carol gets USDC via a claim link	Next payment refused

TECHNICAL EXECUTION

The limit lives on Stellar, not on our server.

Claude, or any MCP app
OAuth 2.1 with PKCE. A connection alone can spend nothing.

Nod wallet service
Cloudflare Worker. Holds only each app's key, reserves before signing, relays and pays every fee.

Your OpenZeppelin smart account

```
rule 0 · your passkey · admin and approvals
rule n · Claude's key · ≤ 5.00 USDC until T
```

Circle USDC
Stellar Asset Contract, settled on testnet

A 5.00 allowance spends exactly 5.00. One stroop more is rejected. `c0f6f923`

An over-limit transfer forced onchain is rejected by the account itself. `3daf4b00`
Error #3221, spending limit exceeded

Your passkey signs the exact payment. Nod cannot change the recipient or amount. `fbfd200c`
20 USDC approved with a WebAuthn signature

Ask twice, pay once. Receipts only after settlement. `idempotency keys`

Nod pays every network fee. The owner holds 0 XLM. `relayer as source`

TRACK ALIGNMENT · STELLAR COMPOSED

Nine Stellar building blocks, one wallet.

OpenZeppelin smart account One contract account per wallet	**OpenZeppelin spending-limit policy** The allowance, N USDC until T	**Passkeys** WebAuthn verifier on the secp256r1 host function
Circle USDC Through the Stellar Asset Contract	**x402 facilitators** x402.org and OpenZeppelin Channels, never our own	**MCP and OAuth** A connector in Claude, any MCP app
SAC trust (CAP-73) Trustline and payment in one transaction	**Sponsored reserves** Each app's payer account holds 0 XLM	**Built-in DEX** The paid market brief reads the XLM/USDC book

> **“Agents that spend: Give an agent a wallet … set its limits with OpenZeppelin's policy library, and let it pay for APIs in stablecoins … over x402 or MPP.”**
>
> Stellar Composed, first idea

> **“No seed phrases, no gas, no jargon. Spending limits … live in the wallet.”**
>
> Stellar Composed, wallets idea

INNOVATION

Four ideas behind Nod.

THE POCKET MODEL

A smart account pays any x402 seller today.

Hosted facilitators reject smart accounts. Claude's key fills its own small account under the policy, then pays.

`your account` → policy → `pocket` → stock x402 client → `x402.org`

CONNECT = ALLOWANCE

Connecting an AI app installs its allowance onchain.

The OAuth consent page and the passkey signature are one step. One rule, expiry and revoke per app.

ASK, DON'T REJECT

An over-allowance payment becomes a passkey approval.

Most agent wallets just block it. Nod sends Claude a link, and your nod covers that exact recipient and amount.

CLAIM LINK

Pays people who have never held USDC.

Carol signs once. The USDC contract's trust function and the transfer run in one invocation.

HACKATHON PROGRESS · 25 OCTOBER, UTC

Built in one day, with a transaction for every claim.

10:55
Research, pinned versions, live probes

11:25
Product spec, first commit

11:25–12:20
Five spikes, each claim proven on testnet

12:15
Four build streams on one main branch

13:38
First end-to-end run on the real chain

14:30
Live on testnet with a public URL

130+
commits, all on main, from 25 October

117
testnet transactions linked in the spikes and evidence

5
spikes that de-risked every hard part first

0
facilitators of our own

MARKET AND ECOSYSTEM

Every AI app is a door.

Distribution through MCP

Any MCP app can connect with one URL. Claude today, more hosts as their payment policies allow.

Buyers for Stellar's x402 sellers

Stellar builders have shipped paid APIs and paid MCP tools. They need buyers whose owners will let them spend.

People first, then teams

One allowance per AI app today. Next, a company card for every agent: limits, approvals and receipts.

Ready for a mainnet pilot

Every contract Nod uses is already on mainnet with identical WASM, except our 920-byte router. A full allowance lifecycle costs about 0.6 XLM at mainnet rent.

TRY IT

Three minutes, in your own Claude.

SITE

`testnet.trynod.si`

CLAUDE CONNECTOR

`testnet.trynod.si/mcp`

NO CLAUDE? WATCH A SAMPLE AGENT

`testnet.trynod.si/sample`

"Buy the market brief"
0.10 USDC, no prompt

"Send Alice 2 USDC"
2.90 left of the 5.00 allowance

"Pay Bob 5 USDC for dinner"
More than what is left, so it needs your nod

New wallets get 10 testnet USDC. Full steps in evidence/JUDGE-TEST-PATH.md.

What's next

- Mainnet pilot with one AI app
- Recovery passkey and shared wallets
- Cash-out through a Stellar anchor
- Task budgets and merchant allowlists
- Agent-to-agent escrow
- No pocket step once facilitators accept smart accounts

